

(Task 1 of 8) In this tutorial, we will review previously learned content.

0

(Task 2 of 8) What is the result of running this program?

Lispy | 

Lispy [Run  <pre>(deffun (foobar n) (defvar m 4) (+ n m)) (+ (foobar 2) m)</pre>	Python <pre>def foobar(n): m = 4 return n + m print(foobar(2) + m)</pre>
---	---

error

2

You predicted the output correctly 🎉🎉🎉

3

`(+ (foobar 2) m)` is evaluated in the top-level block, where `m` is not defined. So, this program errors.

Click [here](#) to run this program in the Stacker.

(Task 3 of 8) What is the result of running this program?

Lispy | 

Lispy [Run  <pre>(defvar v1 (mvec 23)) (defvar v2 v1) (vec-set! v1 0 45) v2</pre>	Python <pre>v1 = [23] v2 = v1 v1[0] = 45 print(v2)</pre>
---	---

`#(45)`

5

You predicted the output correctly 🎉🎉🎉

6

The program creates a one-element vector (the only element being 23) and binds it to both `v1` and `v2`. The 0-th element of the vector is then replaced with 45. So, when the vector is printed as `#(45)`.

Click [here](#) to run this program in the Stacker.

(Task 4 of 8) What is the result of running this program?

Lispy | 

Lispy [Run  <pre>(defvar n (* m 4))</pre>	JavaScript <pre>let n = m * 4;</pre>
---	---

```
(defvar m 3)      let m = 3;
                   console.log(n);
n                 console.log(m);
m
```

error

8

You predicted the output correctly 🎉🎉🎉

9

The first definition tries to bind `n` to the value of `(* m 4)`. To evaluate `(* m 4)`, we need the value of `m`. But `m` is not bound to a value at that moment.

Click [here](#) to run this program in the Stacker.

(Task 5 of 8) What is the result of running this program?

Lispy | 🤖

Lispy [Run ▶]

Scala 3

```
(defvar v1 (mvec 53))    val v1 = Buffer(53)
(defvar v2 (mvec 72 v1)) val v2 = Buffer(72, v1)
(vec-set! v1 0 72)      v1(0) = 72
v2                      println(v2)
```

#(72 #(72))

11

You predicted the output correctly 🎉🎉🎉

12

`v1` is bound to a one-element vector. The only element of this vector is `53`. `v2` is bound to a two-element vector. The elements of this vector are `72` and the one-element vector. The subsequent `(vec-set! v1 0 72)` mutates the one-element vector. Finally, the value of `v2` is printed.

Click [here](#) to run this program in the Stacker.

(Task 6 of 8) What is the result of running this program?

Lispy | 🤖

Lispy [Run ▶]

Python

```
(defvar m1 (mvec 77 77)) m1 = [ 77, 77 ]
(deffun (f m2)           def f(m2):
  (vec-set! m2 0 43)     m2[0] = 43
  (vec-ref m2 0))        return m2[0]
(defvar foo (f m1))      foo = f(m1)
m1                       print(m1)
```

#(43 77)

14

Click [here](#) to run this program in the Stacker.

(Task 7 of 8) What is the result of running this program?

Scala 3

```
(defvar n 5)          val n = 5
(deffun (f1 m)         def f1(m : Int) =
  (deffun (f2)         def f2 =
    (defvar l 4)       val l = 4
    (+ n m l))        n + m + l
  (f2))              f2
(+ (f1 1) 3)         println(f1(1) + 3)
```

17

18

This program binds `n` to 5 and `f1` to a function, and then evaluates `(+ (f1 1) 3)`. The value of `(+ (f1 1) 3)` is the value of `(+ (f2) 3)`, which is the value of `(+ (+ n m 1) 3)`, which is the value of `(+ (+ 5 1 4) 3)`, which is 13.

Click [here](#) to run this program in the Stacker.

(Task 8 of 8) What is the result of running this program?

Python

```
(defvar m (mvec 82 76))    m = [ 82, 76 ]
(vec-set! m 0 m)           m[0] = m
(vec-ref m 1)             print(m[1])
```

20

21

`m` is bound to a vector. `(vec-set! m 0 m)` replaces the 0-th element of the vector with the value of `m`. This is fine, but it's not what we want. We want to bind `m` to the vector itself.

with the vector itself. This is fine because a vector element can be any value, including itself. Besides, the vector is not copied, so the mutation finishes immediately.

Click [here](#) to run this program in the Stacker.

You have finished this tutorial 🎉🎉🎉

Please the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1780302628168